

Trace2Skill: 面向长上下文 EDA 智能体的 验证器引导技能演化

Zijian Du

NVIDIA

Santa Clara, CA, USA

ldu@nvidia.com

Nathaniel Pinckney

NVIDIA

Santa Clara, CA, USA

npinckney@nvidia.com

摘要

复杂 Verilog 设计问题 (CVDP) 对硬件 LLM 智能体构成挑战, 因为解决此类问题需要在大型代码库快照中定位与验证器相关的 RTL、测试平台、包含路径和构建依赖, 进行精确编辑, 并从稀疏的隐藏验证器失败中恢复。我们提出 Trace2Skill, 这是一种无需针对 RTL 进行专门模型微调即可改进硬件智能体的测试时扩展框架。Trace2Skill 不训练新模型, 也不只是采样更多候选解答, 而是将智能体的自然语言技能视为可演化策略。它从反复执行的轨迹中挖掘成功模式与失败模式, 将其转化为稠密诊断信息和 Oracle 经验, 并通过 Oracle-变异器-选择器循环生成任务专用技能, 以指导后续的搜索、编辑、验证和恢复。由于最终的通过/失败标签对于困难失败通常过于粗粒度, Trace2Skill 还支持有界的运行时稠密验证器反馈: 在确保智能体无法访问隐藏测试框架和参考解答的同时, 返回经过净化的功能性观察结果。这种反馈通过收紧技能文本、验证器证据和下游行为之间的协同优化循环, 更好地指导技能演化与智能体执行。在种子 CVDP 智能体无法解决的困难 CVDP 任务上, 包括前沿编码智能体同样失败的任务, 采用稠密验证器反馈的 Trace2Skill 显著提高了任务通过率, 并在此前未解决的任务上取得突破性通过, 同时无需高质量微调数据、专门的 RTL 模型训练或模型权重更新。同一框架提供了一种通用的测试时扩展策略, 可从数字设计推广到其他可验证的 EDA 任务。

KEYWORDS

大语言模型智能体, 电子设计自动化, Verilog, 技能优化, 执行轨迹, 进化算法, 测试时扩展

1 引言

大语言模型正从被动的文本生成器转变为能够推理、检查状态、采取行动并修订工作的工具使用智能体。近期系统将推理与动作交错进行 [1], 无需更新权重即可从执行反馈中学习 [2], 在长时间跨度上积累可复用技能 [3], 并为软件工程提供智能体—计算机接口 [4]。多智能体和基准研究进一步表明, 工具使用、角色结构和真实交互环境正成为智能体研究的核心 [5–8]。

硬件设计对这一进展提出了挑战。既有研究已经推进了 RTL/代码生成基准 [9–12]、芯片领域对齐与推理适配 [13, 14], 以及由编译器、仿真器、检索或波形引导的 Verilog 智能体 [15, 16]。与本文最相关的是, ACE-RTL 将基于大规模 RTL 数据训练的 RTL 专用生成器, 与前沿模型反思、逐问题上下文演化和并行测试时扩展相结合, 在完整 CVDP 类别上取得了很高的智能体通过率; 据其报告, 在四个 CVDP 类别上, 并行扩展将首次成功所需的平均迭代次数从 11.30 降至 4.08 [17]。这些系统展现出明确潜力, 但 CVDP 也暴露了一种更困难的恢复场景: 长上下文、多轮任务要求智能体导航代码库、定位缺陷、集成功能、编辑 RTL、调用工具, 并从验证失败中恢复。许多此类任务会使前沿智能体失败, 且几乎不提供可供智能体改进的学习信号。

Trace2Skill 研究面向这一恢复场景的测试时扩展。它不微调模型权重, 而是演化注入固定 CVDP 智能体

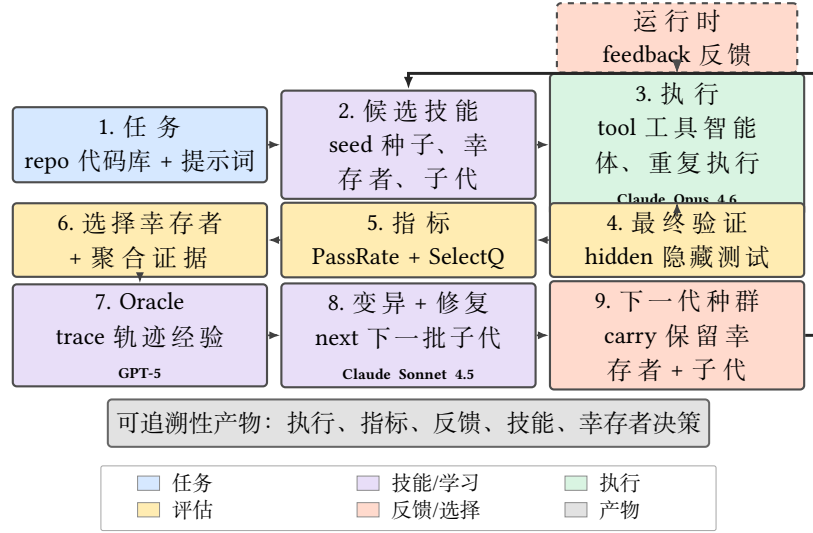


图 1: Trace2Skill 端到端流程。颜色按图例所示区分组件角色；实线箭头表示主要执行/评估路径，粗回返箭头表示技能演化进入下一代种群，虚线双向边表示可选的运行时反馈。模型标签标识由 LLM 驱动的阶段。

的自然语言技能。该技能是一项指导搜索、编辑、验证和恢复的策略产物。每批推出完成后，Trace2Skill 从执行轨迹中挖掘通过模式与失败模式，请 Oracle 总结可操作的经验，并使用进化变异器和选择器生成下一项任务特定技能。运行时稠密验证器反馈是一种可选的黑盒工具，可在推出期间提供额外学习信号：它返回经过净化的功能性观察，同时使隐藏执行框架和参考解保持不可访问。这可以让失败更具可操作性，但在验证计算量较大时可能带来额外开销。

因此，我们的目标与 ACE-RTL [17] 互为补充。ACE-RTL 主要探究经过训练的 RTL 专用模型和逐问题演化上下文能在多大程度上提升广泛 CVDP 类别上的通过率。Trace2Skill 则提出一个不同的问题：当种子 CVDP 智能体失败时，开放智能体能否从执行轨迹中挖掘有用技能，而无需微调模型，包括那些同样会使前沿编程智能体失败的任务？适配单元不仅是当前 RTL 候选或问题上下文，还包括支配未来推出的可复用技能文档；相应地，评估聚焦于困难失败恢复、稠密行为信用分配，以及学得技能内容与智能体执行质量之间的关系。

这些观察塑造了我们的设计。前沿编程智能体是强大的黑盒基线，但开放 CVDP 智能体会暴露轨迹、

验证器结果和技能文本，使逐任务适配可解释、可调节。由于稀疏的通过/失败标签对于困难失败而言过于粗糙，我们设计了同时覆盖技能质量和智能体执行质量的细粒度指标，作为代理优化目标，并表明它们与稀疏通过/失败信号高度相关。最后，困难 CVDP 任务具有异质性：共享技能的迁移并不可靠，这促使我们采用逐任务演化，而不是多任务协同优化。

本文作出以下贡献：

- **Trace2Skill**：一种无需训练的框架，在不更新模型权重的情况下，将 CVDP 执行轨迹转化为任务专用的智能体技能。
- 一种分解式稠密质量信号，通过将技能内容与智能体执行行为分离，对失败执行进行排序。
- 一个开放、可解释的 CVDP Trace2Skill 智能体，能够解决基线 CVDP 智能体失败的困难任务，包括 Claude Code 和 Codex 同样失败的任务。

2 方法

图 1 给出了所有配置使用的执行循环。确定性执行框架组件加载任务、注入技能、强制执行运行时反馈策略、运行隐藏验证、计算指标、聚合逐任务证据并记

录产物；LLM 驱动模块负责推出、Oracle 总结和技能变异。

2.1 执行与反馈

每个 CVDP 任务提供一个代码库快照和目标。基线推出注入特定于 CID 的种子技能；优化推出则注入当前任务特定的存活技能。我们通过 **NeMo Gym** 运行这些工具智能体推出：头服务器启动已提交的运行，智能体交替进行模型调用和资源服务器工具执行，资源服务器则返回工具观察和最终验证器奖励，如附录图 6 所示。随后，一个 Claude Opus 4.6 工具智能体检查文件、编辑 RTL、编译和仿真，并在达到 30 轮预算或完成任务后停止。隐藏验证器给出正式的最终通过/失败标签。在稠密反馈配置中，智能体还可以调用 `verify_feedback`，这是一项在私有工作区副本上运行、有预算限制的黑盒验证器。它会返回经过净化的功能性反馈，同时隐藏参考解、隐藏执行框架文件、原始 cocotb 源码/日志、隐藏路径和 Docker 配置。

2.2 指标

智能体经常无法完成困难的长上下文 CVDP 任务，导致许多执行的得分为零。因此，我们将最终验证器通过率作为最终目标，但使用精心设计的稠密信号来驱动粒度更细的学习过程。各组件的稠密分数会被截断到 $[0, 1]$ ，并且所有任务使用相同权重。

2.2.1 PassRate: 最终正确性。

$$\text{PassRate} = \frac{1}{R} \sum_{r=1}^R p_r. \quad (1)$$

其中， $p_r \in \{0, 1\}$ 是第 r 次重复执行的最终隐藏验证器结果： $p_r = 0$ 表示智能体提交的最终解答未通过验证，而 $p_r = 1$ 表示其提交的解答通过了隐藏验证器。启用运行时稠密反馈时，它绝不会被计入最终正确性；其作用仅是帮助智能体选择后续编辑。

2.2.2 SkillQ: 技能内容质量。

SkillQ 在未来执行之前对修复后的技能文本进行评分。它衡量子代技能能否保留父代指导，并以可执行、智

能体可见的形式整合 Oracle 经验：

$$Q_{\text{skill}}^{\text{raw}} = 0.35L + 0.30G + 0.10R_p + 0.15A_{\text{act}} + 0.05V_s + 0.03N + 0.02D. \quad (2)$$

$$Q_{\text{skill}} = Q_{\text{skill}}^{\text{raw}} (0.55 + 0.45V_s) M_{\text{keep}}. \quad (3)$$

其中， $Q_{\text{skill}}^{\text{raw}}$ 是未经门控的内容分数； L 表示关键经验覆盖率， G 表示证据依据， R_p 表示父代规则保留度， A_{act} 表示可执行性， V_s 表示安全有效性， N 表示非冗余性， D 表示保守的变异规模。 M_{keep} 是一个保留门：它会惩罚丢弃从以往执行中学习到的关键 Oracle 指令的提案。隐藏测试框架泄漏、不可用工具建议、矛盾和语义下限回退会被修复；若修复后仍然存在，候选项将无法保留。总体而言，SkillQ 偏好在保留上一代指导并排除有害或违反规则的的建议的同时，加入当前一代有依据经验的子代。

2.2.3 AgentProgressQ: 执行过程质量。

AgentProgressQ 衡量智能体在一次执行中实际完成了什么，即使最终验证器失败也会进行衡量。它不直接对技能文本评分；而是衡量注入技能所引导的智能体行为：

$$F_{\text{base}} = 0.40V + 0.20X + 0.15H + 0.15E + 0.10\eta. \quad (4)$$

$$F_{\text{progress}} = F_{\text{base}} (0.55 + 0.45P_{\text{path}}). \quad (5)$$

各项分别为： V ，验证器或部分验证器进展； X ，从检查到编辑/编译/仿真所达到的执行阶段； H ，与可见测试框架约定的一致性； E ，编辑质量； η ，轮次效率；以及 P_{path} ，针对编辑或验证目标错误的惩罚。我们观察到，相同技能下的重复执行存在不理想的随机行为，因此重复测量应惩罚不稳定的进展。对于相同候选技能下的重复执行，我们使用置信下界汇总：

$$F_{\text{LCB}} = \max \left(0, \bar{F}_{\text{progress}} - 1.96\sigma/\sqrt{R} \right). \quad (6)$$

$$Q_{\text{agent}}^{\text{prog}} = 0.80F_{\text{LCB}} + 0.20\bar{F}_{\text{progress}}. \quad (7)$$

其中， R 是候选技能的重复执行次数， σ 是其 F_{progress} 分数的标准差。LCB 项降低了单次幸运执行主导选择的概率。从统计角度看，LCB 项估计平均进展的不确定性，因此其惩罚按 $\sigma/\sqrt{R}$ 缩放。下文的 AgentVarianceQ

表 1: 在 CVDP 基线智能体零通过的 8 个困难任务上进行的四配置消融。所有条件均使用相同的任务集、执行模型、30 轮上限和最终隐藏验证器。

配置	智能体	EA 调度	运行时反馈	预算	目的
C1	CVDP 基线	无	仅最终稀疏验证器	评估: $8 \times 4 = 32$ 次执行	种子技能基线。
C2	CVDP 基线	无	稠密反馈 (每次执行 ≤ 3 次调用) 加最终验证器	与 C1 相同	带稠密反馈的种子技能。
C3	CVDP Trace2Skill	5 代 EA; 每代 4 个技能; 每个技能 4 次重复执行	与 C1 相同	优化: $8 \times 5 \times 4 \times 4 = 640$ 次执行	稀疏奖励下的技能演化。
C4	CVDP Trace2Skill	与 C3 相同	与 C2 相同	与 C3 相同	稠密奖励下的技能演化。

统一结果指标: 尝试执行的通过率、解决的任务数/8, 以及 AgentProgressQ。

则刻画执行本身的不稳定性: AgentProgressQ 在优化信号内部惩罚不确定性, 而 AgentVarianceQ 报告执行行为的随机程度。

2.2.4 AgentVarianceQ: 重复稳定性.

$$Q_{\text{agent}}^{\text{var}} = 1 - \min(1, \sigma/0.30). \quad (8)$$

该诊断只使用执行行为。值较高表示相同技能下的重复尝试表现一致; 值较低则警示代际均值可能反映随机的长时程变化。当通过率相同时, 更高的 AgentVarianceQ 表示更强的行为一致性, 因此更适合作为诊断性选择信号。

2.2.5 SelectQ: 演化幸存者分数。

SelectQ 是用于选择下一个父代技能的分数:

$$\text{SelectQ}(S) = \begin{cases} -1, & B(S) = 1, \\ \text{PassRate}(S) + \epsilon U(S), & B(S) = 0, \end{cases} \quad (9)$$

$$U = 0.60F_{\text{LCB}} + 0.20\bar{F}_{\text{progress}} + 0.20Q_{\text{skill}}. \quad (10)$$

$$\epsilon = 0.49/\max(R, N_{\text{task}}, 1). \quad (11)$$

$B(S) = 1$ 标记无效或发生语义回退的技能。较小的 ϵ 使 SelectQ 以通过为主导: 多一次隐藏验证器通过总会超过稠密决胜指标, 而 AgentProgressQ 和 SkillQ 则对通过率相同的候选项进行排序。尽管 SkillQ 与智能体质量项分别测量, SelectQ 仍将它们组合起来, 因为技能内容与执行行为是耦合的: 语义上合理的技能只有在能带来更好的智能体执行时才有用。经验上, 仅按 SkillQ 选择可能保留整洁但在行为上停滞的指导,

而仅按 AgentProgressQ 选择则可能追逐嘈杂的执行结果。因此, SelectQ 将语义技能质量与对重复执行稳健的智能体进展相结合, 既要求指导有效, 又偏好能够改进执行的技能。

2.3 技能演化与可追溯性

Trace2Skill 仅在当前目标任务内部聚合证据。GPT-5 Oracle 将任务局部的执行转化为关于有用行为和重复失败的经验。Claude Sonnet 4.5 变异器根据幸存者技能、Oracle 经验、累积经验库以及工具/可见性契约提出子代技能。确定性后处理会修复或挽救有用提案, 同时移除不安全或缺乏依据的指导。下一代在相同执行协议下评估被保留的父代和修复后的子代; 附录算法 1 给出了形式化更新。所有阶段都会写入产物: 原始轨迹、验证器输出、反馈调用、稠密指标、Oracle 摘要、变异器提案、修复后的技能和幸存者决策。

3 实验设置

首先, 我们在 24 个公开、可使用 OSS 运行的长上下文 CVDP 任务上, 使用未经优化的 CID 种子技能建立 CVDP 基线智能体, 并在相同的隐藏 CVDP 验证器下与 Codex 和 Claude Code 进行比较。这 24 个任务分别包含来自 cid003 (规范到 RTL)、cid004 (RTL 优化)、cid005 (代码补全) 和 cid016 (RTL 错误修复) 的六个任务; 它们不使用商业 EDA 工具, 而是使用 Docker、Icarus Verilog (iverilog/vvp)、与 cocotb 兼容的验证

器镜像以及 CVDP 资源服务器。其次，我们选择附录表 4 中以粗体显示的八个种子 CVDP 失败任务，研究逐任务 Trace2Skill 演化和运行时稠密验证器反馈能否改善 CVDP 智能体在这些困难失败上的表现。同一附录表还记录了 Claude Code 和 Codex 是否也未能解决这些任务。

3.1 配置

24 任务基线在相同的隐藏 CVDP 验证器下，比较了使用未经优化种子技能的 CVDP 智能体、Claude Code 和原生 Codex。CVDP 智能体使用带有固定 CID 种子技能的 CVDP 测试框架，不采用 EA 或稠密反馈；前沿智能体使用各自的测试框架，且不会获得我们的任何技能。对于包含 8 个困难任务的子集，表 1 展示了我们消融的四种配置：C1 是失败的 CVDP 基线智能体，C2 增加稠密反馈，C3 增加使用稀疏最终奖励的逐任务 Trace2Skill，C4 则将逐任务 Trace2Skill 与稠密反馈相结合。

CVDP 执行采用 Claude Opus 4.6，温度设为 0.2。Oracle 汇总采用 GPT-5，温度设为 0.0；变异采用 Claude Sonnet 4.5，温度设为 0.35，随后进行确定性修复。C1/C2 对每个任务重复四次，共进行 32 次基线尝试。C3/C4 使用五代演化，每代四个候选技能，每个候选重复四次：每个任务进行 80 次优化执行，每种 EA 配置共进行 640 次。我们直接报告优化执行本身的 C3/C4 结果，因为本研究关注的是在相同困难任务上的逐任务适应，而非训练/测试技能泛化。

稠密反馈执行每次最多可调用三次 `verify_feedback`（我们首先评估匹配的公开 OSS CVDP 子集：cid003、cid004、cid005 和 cid016 各六个任务。这建立了进行任何 Trace2Skill 适应之前，未经优化的 CVDP 智能体基线。

3.2 AgentQ 与通过/失败相关性

在使用 AgentQ 作为 EA 的平局判定指标之前，我们先检查它是否与种子技能基线推出的最终稀疏成功一致。由于该校准在不使用 EA 的 CVDP 智能体上运行，因此仅采用智能体行为进展：单次推出使用 F_{progress} ，重

复测量使用 $Q_{\text{agent}}^{\text{prog}}$ 。已完成的 96 次基线推出呈现出较高的点二列相关系数 ($r = 0.90$) 和 0.982 的 AUC，表明当困难任务仅提供完全相同的零通过奖励时，该代理指标对稀疏通过/失败结果的跟踪足够紧密，可作为优化目标。

图 2 检验该代理指标是否指向真实的通过/失败行为；它并不取代隐藏验证——后者仍是 SelectQ 中的首要项——但它能提供差异化指导，推动智能体最终通过验证器。

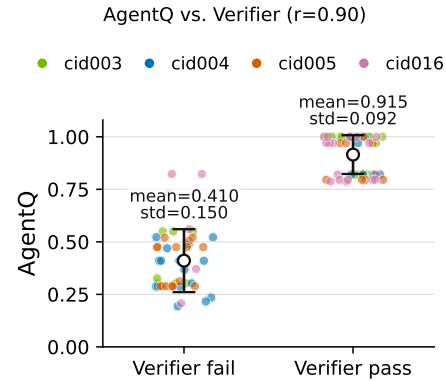


图 2: 96 次已完成的 OSS 种子技能基线推出上的 AgentQ 代理指标，按隐藏验证器结果分组，并按 CID 着色。每个点代表一次推出。

4 结果与分析

4.1 基线智能体比较

表 2: 按 CID 划分的匹配公开 OSS CVDP 基线结果。该子集包含测试框架无需商业 EDA 工具的公开长上下文智能体 CVDP 任务, 每个可在 OSS 环境运行的 CID 采样六个任务。

CID	类别	CVDP 智能体	Claude Code	Codex
cid003	规格到 RTL	4/6 (66.7%)	5/6 (83.3%)	4/6 (66.7%)
cid004	RTL 优化	2/6 (33.3%)	3/6 (50.0%)	2/6 (33.3%)
cid005	代码补全	1/6 (16.7%)	2/6 (33.3%)	2/6 (33.3%)
cid016	缺陷修复	2/6 (33.3%)	2/6 (33.3%)	4/6 (66.7%)
总体	OSS 子集	9/24 (37.5%)	12/24 (50.0%)	12/24 (50.0%)

表 2 显示, 种子技能 CVDP 智能体解决了 9/24 个任务, 而 Claude Code 和 Codex 各解决了 12/24 个任务。前沿智能体是更强的黑盒通过率基线, 但 CVDP 智能体暴露了测试时扩展所需的轨迹、技能、工具和验证器结果。按 CID 分解的结果同样很有启发性: 在三个智能体上汇总后, Spec2RTL 的解决率最高 (13/18), 而错误修复 (8/18)、RTL 优化 (7/18) 和代码补全 (5/18) 更低。这表明, 智能体已经具备相当强的 RTL 编程能力, 但当成功需要在长上下文代码库中定位、编辑现有 RTL、调试失败, 并保持与周边代码兼容时, 性能会下降。附录表 4 中的粗体任务是用于 C1-C4 消融的八个种子 CVDP 失败任务; 同一表格显示, 该集合既包括所有智能体都失败的任务, 也包括前沿智能体在没有我们的轨迹适应基础设施时成功解决的任务。

4.2 困难任务上的最终评估

接下来, 我们评估测试时扩展能否解决种子 CVDP 智能体失败的八个困难任务。C1 是仅使用最终稀疏验证的种子技能 CVDP 智能体。C2 增加运行时稠密验证器反馈, 但不进行技能演化。C3 在稀疏最终奖励下增加逐任务 Trace2Skill EA。C4 将相同的 EA 预算与运行时稠密验证器反馈结合。这不是训练/测试泛化实验: 先前的共享技能测试表明, 异构任务之间的迁移较弱, 因此每种条件都在相同的困难任务集上评估。

表 3 给出了最终结果视图: C4 表现最强, 以 33.6% 的通过率解决了 6/8 个困难任务, 并取得最高的平均

表 3: 相同 8 个困难任务上的 C1-C4 比较。

配置	EA	稠密	推出数	通过率	解决数/8	AgentQ
C1	否	否	32	0.0%	0	0.34
C2	否	是	32	12.5%	2	0.42
C3	是	否	640	27.0%	3	0.523
C4	是	是	640	33.6%	6	0.566

AgentQ。单独使用稠密反馈对采用固定种子技能的智能体有所帮助, 但效果仍有限 (C2: 2/8, 12.5%)。稀疏 Trace2Skill 解决了更多任务 (C3: 3/8, 27.0%), 而在相同 EA 预算下加入稠密反馈又额外解决三个任务, 并将通过率提高 6.6 个百分点 (C4: 6/8, 33.6%)。因此, C1→C2 隔离了种子技能智能体的稠密验证器反馈效果, C1→C3 隔离了稀疏最终奖励下逐任务 Trace2Skill 的效果, 而 C3→C4 则是在 EA 预算匹配条件下最纯粹的稠密反馈比较。该结果支持核心主张: 稠密反馈与逐任务技能演化结合时最为有用, 而不应仅被视为固定种子技能智能体中的一项额外工具。

4.3 质量指标

图 3 将 C3 和 C4 基于产物得到的稠密指标可视化, 包括跨代的 SelectQ、SkillQ、AgentProgressQ 和 AgentVarianceQ。该聚合视图与最终通过表互为补充: SelectQ 是进化存活者得分, SkillQ 总结学得技能的内容质量, AgentProgressQ 总结对重复具有鲁棒性的推出执行质量, 而 AgentVarianceQ 报告重复稳定性。长时间跨度、多轮 LLM 智能体推出中出现可见的不确定性属于预期现象: 即便技能提示词完全相同, 也仍可能诱导出不同的搜索、编辑和验证轨迹, 因此我们使用重复样本估计平均行为, 而不依赖单次运行结果。我们对有效的候选技能测量进行聚合, 排除无效或导致语义下限回退的候选, 使趋势反映可行的存活者选择, 而不是被拒绝的提案。跨代观察可见, C4 的 SelectQ 和 AgentProgressQ 均高于 C3; SkillQ 在二者共享的种子代持平, 随后几代则由 C4 取得更高值。这表明, 运行

时稠密反馈同时改善了学得技能内容和智能体的执行行为，而不只是改变最终的稀疏通过次数。

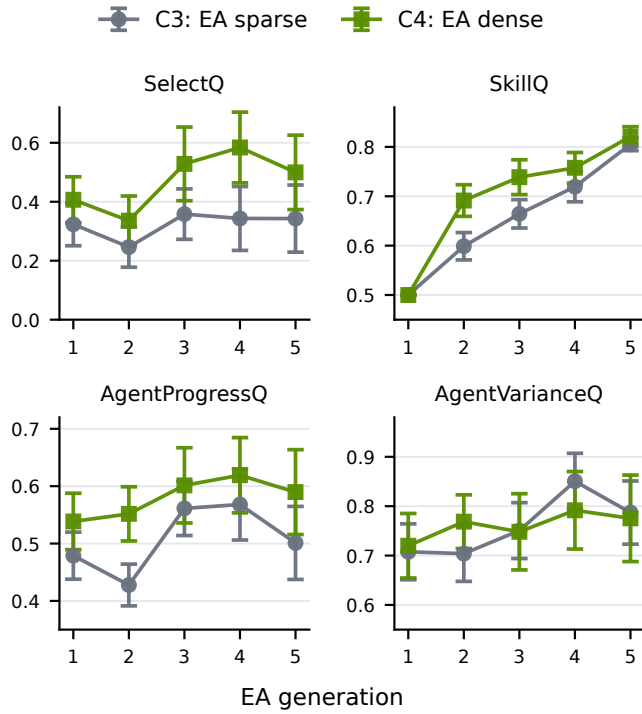


图 3: 8 个困难 CVDP 任务上 C3 与 C4 的质量动态。各点表示代级均值；误差条表示有效候选技能测量上的均值标准误。

图 4 给出了任务级结果视图，作为稠密指标的补充。每行对应一个困难任务，按 CID 分组，每个单元格聚合该任务和条件下的所有 EA 执行。高亮的 C4 单元格表示取得突破的任务：C3 没有任何验证器通过，而 C4 至少取得一次通过。

综合来看，图 3 和图 4 将过程质量与验证器结果联系起来。热图显示，C4 并非在每个任务上都一致更优；其价值在于扩大了可解决的困难任务集合，尤其是 verilog_pcie_0004、802.11a_0001 和 NoobsCpu-8bit_0002。其机制是技能与智能体的协同适应：验证器信号直接塑造执行行为，而生成的轨迹又为 Oracle 和变器提供更明确的下一代经验，从而形成闭环的持续优化循环。

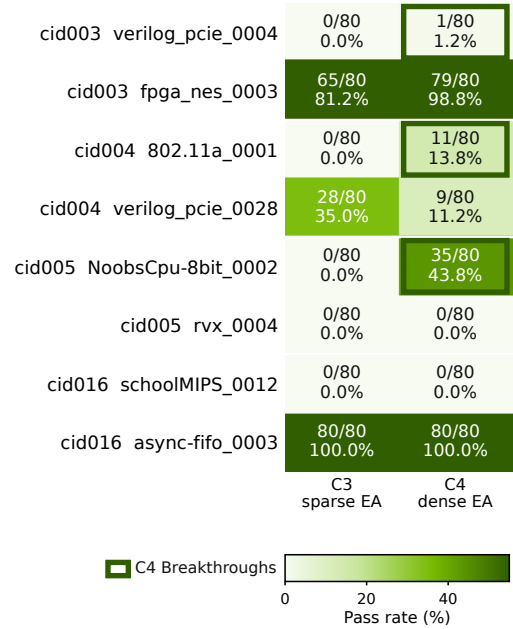


图 4: 8 个困难任务上的任务级 C3/C4 验证器结果。单元格显示五代 × 四个候选技能 × 四次重复执行中的通过次数和通过率。

4.4 任务级通过率

图 4 中加框的单元格是仅 C4 取得的突破。在 C3 中，这些任务的验证器通过次数为零，且没有运行时稠密反馈调用。在 C4 中，PCIe 达到 1/80 次通过，802.11a 达到 11/80 次，NoobsCpu 达到 35/80 次。轨迹显示了不同程度的稠密反馈依赖。对于 PCIe，通过的执行首先通过了本地脉冲合并测试，随后反馈揭示了隐藏的功能失败；进一步编辑后，后续反馈调用通过了寄存后的 pulse_out 变体。对于 802.11a，11 次通过执行中有 9 次围绕 MAX_LENGTH=192 Viterbi 测试和部分失败调用了反馈；另有两次通过执行没有调用，因此保守的解读是：反馈改善了测试框架一致性和 Oracle 经验，但并非每次单独通过都必不可少。对于 NoobsCpu，关联最强：全部 35 次通过执行都使用了反馈，代表性轨迹显示，本地 iverilog/vvp 成功后出现验证器编译失败，随后修复包含文件处理、显式文件列表、asserts.v 和 noobs_cpu_defines.vh，直至反馈返回通过。因此，稠密反馈将隐藏的最终失败转化为执行过程中可操作的

纠正措施, 以及更明确的 Oracle/变异器经验, 同时仍保留了可见的任务难度和智能体随机性。

4.5 局限性

- **智能体。**长时程智能体在相同技能下可能采用不同的搜索、编辑和验证路径, 因此, 即使最佳幸存者技能有所改善, 平均指标也未必单调提升。未来工作应更好地刻画技能文本与稠密反馈如何影响执行方差和任务成功率。
- **计算。**Trace2Skill 需要大量计算。30 轮的时程为智能体提供了恢复空间, 但也增加了执行方差、验证器流量、实际耗时和 token 成本。因此, 通过更合理的验证器调度、更早停止和更强的程序性指导来减少所需轮次预算, 是一个核心系统方向。
- **基准。**本文研究的 OSS CVDP 子集很有价值, 但仍集中于 Verilog 相关编码任务。要取得更广泛的进展, 需要 EDA 社区贡献超越 RTL 编码的开放、可执行、长上下文基准, 使智能体方法能够在综合、时序、功耗、形式验证和全流程工程任务上进行比较, 从而惠及更广泛的生态系统。

5 结论

Trace2Skill 通过演化注入固定 CVDP 智能体的自然语言技能, 将执行轨迹转化为硬件智能体的测试时优化信号。在种子 CVDP 智能体失败的八个任务上, 包括 Claude Code 和 Codex 同样失败的任务, 采用稠密验证器反馈的完整 Trace2Skill 配置解决了 6/8 个任务, 隐藏验证器通过率达到 33.6%; 相比之下, 种子智能体为 0/8, 仅使用稠密反馈为 2/8, 稀疏技能演化为 3/8。证据表明存在技能与智能体的协同适应: 稠密验证器观察能够改善执行行为, 而由此产生的轨迹又为 Oracle 和变异器提供更好的经验, 用于下一代演化。由于该循环只需要工具轨迹、可检查的技能文本和可靠奖励, 它可以补充 RTL 专用模型训练, 并扩展到其他可验证的 EDA 与工程任务。

致谢

感谢 Joshua Mabry、Arti Jain 和 Mark Ho 提供富有帮助的讨论与反馈。

REFERENCES

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [2] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [3] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.
- [4] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [5] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkan Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling*, 2024.
- [6] Sirui Hong, Xiaowu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, and Chenglin Wu. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- [7] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kai Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.
- [8] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.
- [9] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. Verilogeval: Evaluating large language models for verilog code generation. In *IEEE/ACM International Conference on Computer-Aided Design*, 2023.
- [10] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. In *Asia and South Pacific Design Automation Conference*, 2024.

- [11] Christopher Batten, Nathaniel Pinckney, Mingjie Liu, Haoxing Ren, and Brucek Khailany. PyHDL-Eval: An LLM evaluation framework for hardware design using python-embedded DSLs. In *ACM/IEEE Symposium on Machine Learning for CAD*, 2024.
- [12] Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. Comprehensive verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification, 2025.
- [13] Chenhui Deng, Yunsheng Bai, and Haoxing Ren. ChipAlign: Instruction alignment in large language models for chip design via geodesic interpolation. In *ACM/IEEE Design Automation Conference*, 2025.
- [14] Chenhui Deng, Yun-Da Tsai, Guan-Ting Liu, Zhongzhi Yu, and Haoxing Ren. ScaleRTL: Scaling LLMs with reasoning data and test-time compute for accurate RTL code generation. In *ACM/IEEE Symposium on Machine Learning for CAD*, 2025.
- [15] Yun-Da Tsai, Mingjie Liu, and Haoxing Ren. RTLFixer: Automatically fixing RTL syntax errors with large language models. In *ACM/IEEE Design Automation Conference*, 2024.
- [16] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. VerilogCoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree-based waveform tracing tool. In *AAAI Conference on Artificial Intelligence*, 2025.
- [17] Chenhui Deng, Zhongzhi Yu, Guan-Ting Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. ACE-RTL: When agentic context evolution meets RTL-specialized LLMs, 2026.

A 产物附录

本附录给出了 Trace2Skill 流水线使用的紧凑型轨迹依据产物。完整原始轨迹保留在实验目录中；隐藏验证器配套文件、绝对文件系统路径和用户标识符均被省略。

A1. 任务集与基线结果

表 4 给出了用于匹配基线比较的 24 个公开、可使用 OSS 运行的长上下文 CVDP 任务。粗体行是用于 C1–C4 Trace2Skill 消融的八个种子 CVDP 失败任务。结果列显示 Claude Code 和 Codex 是否也未能解决这些任务；失败说明汇总所有智能体均失败的任务，或标记选定的仅种子智能体失败任务。

表 4: 用于匹配基线比较、可在公开 OSS 环境运行的长上下文智能体 CVDP 任务。该子集覆盖四个无需商业 EDA 工具即可运行测试框架的公开 CID。粗体任务 ID 表示 C1–C4 消融研究所针对的八个种子 CVDP 失败任务；结果列展示 Claude Code 和 Codex 是否也在相应任务上失败，最后一列给出简要失败说明。

CID	类别	任务 ID	CVDP 智能体	Claude Code	Codex	失败说明
cid003	规格到 RTL	cvdp_agentic_heavy_axi4_lite_0002	√	√	√	–
cid003	规格到 RTL	cvdp_agentic_heavy_corundum_0002	√	√	√	–
cid003	规格到 RTL	cvdp_agentic_heavy_detect_human_faces_0002	√	√	×	–
cid003	规格到 RTL	cvdp_agentic_heavy_fpga_nes_0003	×	√	√	CVDP 种子失败：前沿智能体通过
cid003	规格到 RTL	cvdp_agentic_heavy_fpu_wrappers_0002	√	√	√	–
cid003	规格到 RTL	cvdp_agentic_heavy_verilog_pcie_0004	×	×	×	脉冲合并语义：预期为低电平时 pulse_out 为高电平
cid004	RTL 优化	cvdp_agentic_heavy_802.11a_0001	×	×	×	维特比解码器语义：隐藏输出位不匹配
cid004	RTL 优化	cvdp_agentic_heavy_SoomRV_0002	√	√	×	–
cid004	RTL 优化	cvdp_agentic_heavy_cva6_riscv_core_0005	×	√	×	–
cid004	RTL 优化	cvdp_agentic_heavy_enso_0003	√	√	√	–
cid004	RTL 优化	cvdp_agentic_heavy_pulpino_0004	×	×	√	–
cid004	RTL 优化	cvdp_agentic_heavy_verilog_pcie_0028	×	×	×	未保留 DMA 立即数/控制寄存器语义
cid005	代码补全	cvdp_agentic_heavy_NoobsCpu-8bit_0002	×	×	×	生成的 CPU 未通过 Icarus 构建/展开
cid005	代码补全	cvdp_agentic_heavy_cv32e40p_1_0001	×	×	×	LSU 监视器集成未通过 Icarus 构建/展开
cid005	代码补全	cvdp_agentic_heavy_dual_core_0035	×	×	×	USB 顶层接口/时钟不匹配
cid005	代码补全	cvdp_agentic_heavy_edma_0008	√	√	√	–
cid005	代码补全	cvdp_agentic_heavy_rvx_0004	×	×	×	缺少 RVX 顶层接口信号
cid005	代码补全	cvdp_agentic_heavy_usb_cdc_0002	×	√	√	–
cid016	RTL 缺陷修复	cvdp_agentic_heavy_802.11a_0005	√	×	√	–
cid016	RTL 缺陷修复	cvdp_agentic_heavy_async_fifo_0003	×	√	√	CVDP 种子失败：前沿智能体通过
cid016	RTL 缺陷修复	cvdp_agentic_heavy_deepfreeze_0027	×	√	√	–
cid016	RTL 缺陷修复	cvdp_agentic_heavy_enso_0060	√	×	×	–
cid016	RTL 缺陷修复	cvdp_agentic_heavy_schoolMIPS_0012	×	×	×	时钟分频器和 PC 推进语义失败
cid016	RTL 缺陷修复	cvdp_agentic_heavy_usb_cdc_0017	×	×	√	–

A2. 基于真实轨迹的 Trace2Skill 生成示例

表 5 和 6 对已完成 C4 运行中保存的产物进行了精炼。它们并非用于说明的模拟示例：其中的数值、工具序列和失败模式均取自生成指标、推出诊断、经验库、变异健康日志和评估摘要。第一个示例展示了在 NoobsCpu-8bit_0002 上一次成功的稠密反馈生成；第二个示例展示了运行时验证器反馈如何在 802.11a_0001 上揭示与评估器相关的失败信息。

表 5: 来自 NoobsCpu-8bit_0002 第 5 代、基于真实轨迹的 C4 生成示例。C4 使用逐任务 Trace2Skill，并结合运行时稠密验证器反馈。

产物	记录的轨迹内容	重要性
运行状态	任务 cvdp_agentic_heavy_NoobsCpu-8bit_0002, cid005, C4 稠密反馈 EA, 第 5 代, 种群大小 4, 重复 4 次。该代在 16 次推出中有 9 次通过验证器；最佳个体在 4/4 次重复中均通过。	表 4 中的三个基线智能体在该任务上全部失败，但演化后的 C4 谱系取得了非零且可重复的稀疏通过。
选中的存活者	选中的子代为 child_gen5_1。其日志值分别为 SelectQ 1.104、稀疏 pass@1 1.000、鲁棒效用 0.853、SkillQ 0.894 和 AgentProgressQ 0.820。所选技能不存在关键指令缺失、矛盾或无效技能传播。	选择并非仅依据一次侥幸成功的推出：它在执行有效性检查的同时，综合考虑稀疏成功、鲁棒的重复进展和技能质量。
通过的推出轨迹	一次通过的推出 ind1_r0 调用了 30 次工具。其精简序列为：读取技能和文件、检查模块、写入 vmodel/noobs_cpu.v、运行 iverilog 和 vvp、调用 verify_feedback、再次编辑，将编译/仿真/反馈循环再重复两次，最后通过隐藏验证器。	该轨迹展现了稠密反馈的预期行为：智能体在推出期间使用本地编译/仿真和有界的黑盒验证器反馈，而最终隐藏验证仍作为正式奖励。
进展组成项	代级组成项均值分别为：编辑质量 0.956、效率 0.869、执行阶段 0.956、执行框架对齐 0.872、路径落地 0.403、验证器部分得分 0.563。路径落地诊断仍记录了提示词发现的文件与 vmodel/ 下同名文件之间的影子路径问题。	指标分解很有用，因为单一稀疏通过会同时掩盖成功行为和遗留弱点。在此，C4 解决了任务，但仍暴露出需要分析的路径落地风险。
Oracle 经验	累积经验库保留了具体规则：连线前读取子模块，使用紧凑的 iverilog → vvp 循环，仅编译 asserts.v 一次，优先使用稳定的 -I 包含根目录而非反复修改 include，并使用 verify_feedback 或构建产物来推断预期连线和实例名称。	Oracle 将重复的推出轨迹转化为任务局部的过程记忆，而非泛泛的硬件建议。
演化技能摘录	第 5 代技能明确允许使用 verify_feedback，指示智能体检查 ifetch、idecode、register_file 和 execute，警告重复模块/include 的处理问题，并要求在小幅编辑后执行编译/运行循环。	注入的技能是一项可审计产物。它既保留持久经验，也添加从较早代次中学到的任务特定指导。

表 6: 来自 802.11a_0001 的运行时稠密验证器反馈与变异修复证据。

产物	记录的轨迹内容	重要性
稠密反馈调用	一次第 4 代执行编辑了 Hardware/Receiver/ViterbiDecoder/ViterbiDecoder.v, 对其进行编译并运行本地仿真, 随后调用 verify_feedback。保存的诊断信息记录了最终模式 fail:simulation_or_assertion、阶段 P4-tested、Docker 退出码 1, 以及在 MAX_LENGTH=192 下经过净化的验证器末尾输出, 其中包含 SystemExit: ERROR: Failed 3 of 5 tests。其他第 4 代诊断记录了 3/5 和 5/5 失败, 而成功执行报告 TESTS=5 PASS=5 FAIL=0。	智能体在看不到隐藏测试框架或参考解答的情况下, 获得与评估器相关的功能性观察。这比最终的二元失败信号更稠密, 但仍属于黑盒反馈。
智能体响应	保存的代际提示词和响应明确讨论了 MAX_LENGTH=192 设置、官方运行器的 3/5 或 5/5 失败、192 位输出不匹配, 以及时序或回溯行为。反馈调用后, 采样执行继续进行 RTL 编辑, 并执行更多 iverilog/vvp 检查。	该轨迹证实, 稠密工具确实影响了执行轨迹, 而不只是出现在工具列表中。
Oracle 诊断	代际摘要识别出一种有害模式: 许多失败执行依赖自行编写的测试平台, 例如 ViterbiDecoder_tb2.v、ViterbiDecoder_tb3.v、mini_tb.v 或 debug_tb.v, 从而导致超时或与测试框架参数不匹配。成功执行与官方运行器及 verify_feedback 证据的一致性更好。	稠密反馈帮助 Oracle 区分局部看来合理的行为与符合隐藏验证器的行为, 从而为下一次变异生成更明确的经验。
变异修复	变异健康日志显示, 存在缺陷的子代会被修复, 而非直接拒绝。一个第 3 代修复子代接受了净化器修复 rewrote_harness_execution_to_visible_metadata 和 rewrote_unconfirmed_path_guidance; 其整合覆盖率为 1.000, 而父代覆盖率为 0.625, 修复后缺失的关键指令和矛盾均为零。	在可能的情况下, 有用的语义提案会被挽救; 而隐藏测试框架泄漏、缺乏依据的路径指导或矛盾指令会在技能进入竞赛前被移除。

A3. 原始 EA 代际轨迹缩写

表 7 记录了保存的 C4 产物中一次按原文缩写的代际转换。它遵循实现所使用的同一任务局部链路：任务提示词、执行轨迹、评分、Oracle 摘要、变异器健康状态和下一项注入技能。较长的工具列表经过缩短，隐藏验证器路径或源码则被省略。

表 7: 在 NoobsCpu-8bit_0002 上一次 C4 EA 转换（第 4 代到第 5 代）的原始产物缩写。

阶段	原始产物文本缩写
任务提示词	Create a top-level RTL module named noobs_cpu in /code/vmodel/noobs_cpu.v ... 任务列出的子模块: ifetch、idecode、reg_file 和 execute; 头文件: asserts.v 和 noobs_cpu_defines.vh。交付物: iverilog -g2012 noobs_cpu.v ifetch.v idecode.v register_file.v execute.v。
执行轨迹	rollout_diagnostics.jsonl: gen4 ind2 r0 outcome=pass reward=1.0 phase=P4-tested num_tool_calls=28。工具序列: [cat, ls, ..., echo, rg, iverilog, vvp, verify_feedback, edit, edit, iverilog, vvp, verify_feedback, edit, iverilog, vvp, verify_feedback, show_changes]。写入的文件包括 vmodel/noobs_cpu.v。隐藏验证器末尾输出: test_cpu.cpu_smoke_test PASS; TESTS=1 PASS=1 FAIL=0; 1 passed in 0.14s。
评分	combined_selection_fitness.json: ind2 child_gen4_2 pass@1=0.75 SelectQ=0.818482 robust_utility=0.559034 SkillQ=0.596722 AgentBehaviorQ=0.481760 progress_lcb95=0.432600 progress_mean=0.678400。其他候选选项的 SelectQ 分别为 0.556877、0.294436 和 0.050071; child_gen4_2 成功保留。
Oracle 摘要	oracle_feedback.md: best=0.818 mean=0.430 worst=0.050 rollouts=16 (6 pass, 10 fail)。[cid005] NoobsCpu-8bit_0002 avg=38% (75%, 50%, 25%, 0%) [SOMETIMES SOLVED]。保留: 接线前读取子模块; 采用紧凑的 iverilog -> vvp 循环; 将 asserts.v 恰好编译一次。新增: 受保护路径定位; 协调顶层模块/子模块修复。移除: 优先编辑同基名的 vmodel/ 镜像; 移除断言或为其添加保护。
变异与修复	mutation_handoff.md: gen4_ind2 EFFECTIVE selection +0.523 pass +0.500 progress +0.242。mutation_health.jsonl child_slot=1 parent=child_gen4_2 ok=true has_oracle_feedback=true similarity=0.468 coverage=1.000 parent_coverage=0.792 missing_critical=0 parent_missing=3 remove_violation=0 contradiction=0 repair_attempted=false。
下一项技能	ea_survivor_skill.md: Hard Rules: use only ... verify_feedback; text-only answers ignored; compile then simulate. Protected Path Grounding: edit and validate exact target files ... Do NOT write or validate only same-basename shadow paths. Iverilog Semantics: -I only affects include lookup; pass all required .v files exactly once. Asserts Handling: compile asserts.v exactly once; do not remove asserts。

A4. 执行契约、安全性与遥测

算法 1 给出了当前 EA 循环所实现的紧凑代际更新。父代幸存者始终被纳入下一代种群，子代在评估前经过修复，Oracle 会汇总任务局部的完整种群，使失败子代也能贡献负面经验。

Algorithm 1 标准 EA 循环实现的逐任务 Trace2Skill 演化更新。

Require: 任务 x ; 种群大小 K ; 重复次数 R ; 第 g 代种群 $P_g = [S_0, \dots, S_{K-1}]$, 其中 S_0 是被保留的父代或种子技能; 上一经验库 L_{g-1} 。

Ensure: 下一代未经评估的种群 P_{g+1} 和更新后的经验库 L_g 。

```

1: for  $i = 0$  to  $K - 1$  do
2:    $T_i \leftarrow \{\text{ROLLOUTANDVERIFY}(x, S_i, r)\}_{r=1}^R$ 
3:    $M_i \leftarrow \text{COMPUTEMETRICS}(T_i, S_i)$ 
4:    $q_i \leftarrow \text{SELECTQ}(M_i, S_i, S_0)$ 
5: end for
6:  $S_g^* \leftarrow \arg \max_{S_i \in P_g} q_i$  ▷ 仅在执行后选择幸存者
7:  $\text{WRITEARTIFACTS}(\{T_i, M_i, q_i\}_{i=0}^{K-1}, S_g^*)$ 
8:  $O_g \leftarrow \text{ORACLE}(\{T_i\}_{i=0}^{K-1}, S_g^*, L_{g-1})$  ▷ 汇总所有种群证据
9:  $L_g \leftarrow \text{UPDATELESSONBANK}(L_{g-1}, O_g, \{M_i\}_{i=0}^{K-1})$ 
10:  $H_g \leftarrow \text{BUILDMUTATIONHANDOFF}(O_g, L_g)$ 
11:  $P_{g+1} \leftarrow [\text{CARRYFORREEVALUATION}(S_g^*)]$ 
12: while  $|P_{g+1}| < K$  do
13:    $\hat{S} \leftarrow \text{MUTATOR}(S_g^*, H_g)$ 
14:    $\tilde{S} \leftarrow \text{REPAIRANDSANITIZE}(\hat{S})$ 
15:   if  $\text{MUTATIONHEALTHOK}(\tilde{S}, S_g^*)$  then
16:     将  $\tilde{S}$  追加到  $P_{g+1}$  ▷ 在第  $g+1$  代评估
17:   else
18:     将  $\text{CARRYFORREEVALUATION}(S_g^*)$  追加到  $P_{g+1}$  ▷ 保留父代的回退方案
19:   end if
20: end while
21: return  $P_{g+1}, L_g$ 
22: function  $\text{SELECTQ}(M_i, S_i, S_0)$ 
23:    $u_i \leftarrow 0.60F_{\text{LCB}} + 0.20\bar{F}_{\text{progress}} + 0.20Q_{\text{skill}}$ 
24:   if  $\text{INVALID}(S_i)$  或  $Q_{\text{skill}}(S_i) + \tau < Q_{\text{skill}}(S_0)$  then
25:     return  $-1$ 
26:   end if
27:   return  $\text{pass}@1_i + \epsilon_i \text{clip}(u_i, 0, 1)$ 
28: end function

```

实现说明。代码将选中的幸存者带入下一代重新评估，对同一幸存者进行变异以生成子代，在注入前修复并净化所提出的子代，并将子代选择推迟到这些子代完成自身执行之后。因此，未经评估的子代绝不可能仅凭语义比较就成为下一个父代。

表 8: 通过保存的产物检查的可见性与稠密反馈契约。

契约	基于产物的证据	智能体可见的效果
隐藏验证器隔离	执行契约将隐藏验证器标记为实时执行过程不可访问。稠密反馈在私有副本上运行；隐藏测试框架文件、原始 cocotb 源码、Docker compose 文件、参考补丁和隐藏路径均不会写入实时智能体沙箱。	智能体可以使用 iverilog、vvp，以及稠密配置中的 verify_feedback；但无法检查或编辑隐藏评估器。
净化后的稠密反馈	稠密反馈契约与遥测记录有界摘要，例如通过/失败或最终模式、编译/仿真状态、超时/失败阶段、可用时的部分测试信息，以及净化后的失败上下文。原始隐藏测试框架源码和参考解答会被省略。	反馈类似黑盒功能测试反馈：足以指导下一次 RTL 编辑，但不足以泄露答案。
变异净化器	变异健康日志记录允许使用的工具、可见上下文的路径数量、隐藏验证器状态、对 Oracle 指令的覆盖率、缺失的关键指令、矛盾，以及将隐藏测试框架执行指导改写为可见元数据指导等修复操作。	质量良好但存在缺陷的提案可以得到修复和评估；不安全或缺乏依据的内容会被阻止静默传播。

表 9: 为事后分析保留的遥测产物。

产物类别	代表性文件	支持的分析
运行状态与契约	status.json、run_config.json、execution_contract.json、preflight_report.json	重建分支/配置、任务、代数、种群、重复预算、模型选择、工具可见性和隐藏验证器策略。
执行轨迹	rollout_diagnostics.jsonl、每个个体的 eval_summary_*.json、rollouts_*_agent_metrics.json	恢复工具序列、读写的文件、编译命令、验证器调用、最终通过/失败、路径定位问题和所达到的阶段。
稠密指标与选择	generation_metrics.json、combined_selection_fitness.json、rollout_model_check_*.json	计算 SelectQ、SkillQ、AgentProgressQ、Agent-VarianceQ、pass@1、pass@k、置信区间以及选择幸存者的依据。
学习产物	oracle_feedback.md、lesson_bank.md、mutation_handoff.md、skill_patch_plan.json、best_skill.md、ea_survivor_skill.md	审计执行失败是否被总结为具体经验，以及这些经验是否保留到下一项注入技能中。
安全与修复	skill_sanitization.jsonl、skill_integration.jsonl、mutation_health.jsonl	检查隐藏测试框架泄漏防护、不可用工具移除、矛盾修复和关键指导保留。

A5. 代码库组件图

该实现在研究编排器、内置的 NeMo Gym CVDP 测试框架、技能提示词、优化器模块和运行产物之间分布。其闭环路径为：配置与任务 JSONL → 种子或演化技能 → NeMo Gym 执行服务器 → 工具与验证器轨迹 → 稠密指标与 SelectQ → Oracle 摘要 → 变异器与修复 → 下一代技能。图 5 给出了树状代码库视图，而表 10 将相同组件映射到其流水线角色。生成的缓存和旧版配置未在树中显示。

图 5: 代码库相对路径下的 Trace2Skill 实现及代表性运行产物的树状视图。

```

trace-skill/
|-- orchestration/
|   |-- run_experiment.py
|   |-- configs/mlcad_per_task_oss24_ea/
|       |-- manifest.json
|       |-- cvdp_per_task_*.yaml
|-- nemogym/
|   |-- data/cid_splits_manifest.json
|   |-- nemo_gym/cli.py
|   |-- responses_api_agents/cvdp_agentic_heavy_agent/app.py
|   |-- responses_api_models/openai_model/app.py
|   |-- resources_servers/cvdp_agentic_heavy/
|       |-- app.py
|       |-- context_pruning.py
|-- skills/
|   |-- rollout/*.md
|   |-- optimizer/*.md
|   |-- analysis/*.md
|-- optimizer/
|   |-- run_optimizer.py
|   |-- fitness.py
|   |-- skill_selector.py
|   |-- streaming_oracle.py
|   |-- prompt_mutator.py
|   |-- skill_sanitizer.py
|   |-- skill_integration.py
|   |-- lesson_bank.py
|   |-- telemetry.py
|-- eval/
|   |-- convert_rollouts.py
|   |-- run_analyzer.py
|   |-- run_judge.py
|-- tests/
|   |-- test_standard_ea_flow.py
|   |-- test_dense_progress_fitness.py
|   |-- test_skill_sanitizer.py
|   |-- test_skill_integration.py
|   |-- test_mutation_health.py
|   |-- test_traceability_artifacts.py
|-- paper/mlcad2026_trace2skill/
    |-- appendix/artifact_appendix.tex

<artifact_root>/trace_skill_runs/.../
|-- runs/<task_run>/
|   |-- status.json
|   |-- run_config.json
|   |-- execution_contract.json
|   |-- gen*/
|       |-- rollout_diagnostics.jsonl
|       |-- generation_metrics.json
|       |-- combined_selection_fitness.json
|       |-- oracle_feedback.md
|       |-- mutation_handoff.md (mutation generations)
|       |-- lesson_bank.md
|       |-- ea_survivor_skill.md

```

表 10: 端到端 Trace2Skill 流程使用的代码库相对路径组件。

层	代码库或产物组件	在流程中的作用
实验编排	orchestration/configs/mlcad_per_task_oss24_ea/manifest.json; 逐任务 YAML 文件; orchestration/run_experiment.py。	定义 C1-C4 条件、任务列表、种群、代数、重复次数、模型、输出目录和稠密反馈标志。
CVDP 任务数据	nemogym/data/cid_splits_manifest.json; nemogym/data/ 下的 CID 文件夹; 运行局部的 tasks/*.jsonl。	存储可见任务提示词、代码库上下文、允许使用的工具、种子技能引用, 以及经 OSS 裁剪后实例化的任务材料。实时智能体工作区不包含隐藏验证器附属材料。
NeMo Gym 运行时	nemogym/nemo_gym/cli.py; nemogym/responses_api_agents/cvdp_agentic_heavy_agent/app.py; nemogym/responses_api_models/openai_model/app.py。	启动推出智能体和模型服务端点。CVDP 智能体循环接收技能、调用可见工具、编辑 RTL, 并记录动作轨迹。
CVDP 资源服务器	nemogym/resources_servers/cvdp_agentic_heavy/app.py; context_pruning.py; CVDP 转换脚本。	实现文件工具、iverilog、vvp、最终 verify 和可选的 verify_feedback。稠密反馈在私有副本上运行隐藏验证器, 且仅返回经过净化的功能性观察。
技能与提示词资产	skills/rollout/*.md; skills/optimizer/*.md; skills/analysis/*.md。	推出技能提供 CID/任务指导。优化器技能定义 Oracle、变异、修复和交接提示词。分析技能支持离线轨迹解读。
Trace2Skill 优化器	optimizer/run_optimizer.py; fitness.py; skill_selector.py; streaming_oracle.py; prompt_mutator.py; skill_sanitizer.py; skill_integration.py; lesson_bank.py; telemetry.py。	评估候选技能, 计算稠密指标和 SelectQ, 选择存活者, 总结种群轨迹, 变异生成子代, 修复不安全提案, 更新经验库, 并写入可追溯性产物。
NAT 风格转换与分析	eval/convert_rollouts.py; eval/run_analyzer.py; eval/run_judge.py。	使用 NAT 风格的评估模式, 将原始推出 JSONL 转换为结构化轨迹和行为指标。当前代码库既未内置 NAT 软件包, 也未直接调用它。
运行产物	包含 status.json、run_config.json、execution_contract.json、gen*/rollout_diagnostics.jsonl、generation_metrics.json、combined_selection_fitness.json、oracle_feedback.md、mutation_handoff.md、lesson_bank.md 和 ea_survivor_skill.md 的运行目录。	提供用于事后分析、论文图、通过率表、稠密指标图、技能谱系检查、安全审计和 C4 突破轨迹解释的证据。
验证测试	tests/test_standard_ea_flow.py; test_dense_progress_fitness.py; test_skill_sanitizer.py; test_skill_integration.py; test_mutation_health.py; test_traceability_artifacts.py; 验证器反馈冒烟测试产物。	在启动长时间运行之前, 检查 EA 循环、评分、净化器、提案修复、遥测、提示词卫生和稠密反馈策略是否按预期工作。

A6. NeMo Gym 推出序列

图 6 展开了 CVDP 智能体使用的推出服务器交互过程。头服务器启动一次运行，智能体在模型生成与资源服务器工具执行之间交替进行，资源服务器则返回中间工具结果和最终验证器奖励。

图 6: 一次提交运行的 NeMo Gym 推出序列。

